

Organizzazione Fisica dei Dati (Parte I)

Angelo Montanari

Dipartimento di Matematica e Informatica
Università di Udine

File di record

I dati memorizzati su disco sono organizzati in **file di record**.

Ogni record è una collezione di dati interpretabile come un insieme di fatti relativi a entità, loro attributi e loro relazioni (ad esempio, una tupla può essere memorizzata come un record nel quale il valore di un attributo può essere memorizzato in un campo del record)

Esistono diverse tecniche di memorizzazione fisica dei record di un file su un disco magnetico:

1. **Heap file** (file non ordinato) posiziona i record sul disco in modo non ordinato
2. **Sorted file** (file sequenziale) ordina i record secondo il valore di un particolare campo
3. **Hashed file** usa una funzione di hash per determinare la posizione dei record sul disco

Modalità di accesso alla base di dati

Molte applicazioni che utilizzano basi di dati richiedono di esaminare solo una piccola porzione di esse.

Ogni specifico insieme di dati richiesto da un'applicazione

1. deve essere **localizzato** all'interno della base di dati;
2. deve essere **copiato** in memoria principale, per poter essere elaborato;
3. se sono stati effettuati degli aggiornamenti, i dati aggiornati vanno copiati (**riscritti**) in memoria secondaria.

Record di lunghezza fissa e variabile - 1

Un file è una sequenza di record, usualmente dello stesso tipo.

Se i record di un file hanno tutti la stessa dimensione (in byte), si dice che il file è composto da record di **lunghezza fissa**; altrimenti, si dice che è composto da record di **lunghezza variabile**.

Le ragioni per cui un file può avere record di lunghezza variabile possono essere varie:

- (a) tutti i record del file sono dello stesso tipo, ma uno o più campi hanno una **dimensione variabile** (ad esempio, il campo *NOME* di *IMPIEGATO*);
- (b) tutti i record del file sono dello stesso tipo, ma uno o più campi possono avere **valori multipli** per singoli record; tale campo viene chiamato *campo ripetuto* e un gruppo di valori per tale campo viene chiamato gruppo ripetuto;

Record di lunghezza fissa e variabile - 2

- (c) tutti i record del file sono dello stesso tipo, ma uno o più campi sono opzionali, ossia possono avere un valore per alcuni record, ma non per altri (**file** con campi **opzionali**);
- (d) il file contiene record di più tipi e di dimensioni variabili (**file mixed**). Ciò accade, ad esempio, quando record di tipo diverso sono collocati all'interno degli stessi blocchi (i record *ESAME* relativi ad un determinato studente possono essere posizionati subito dopo il record *STUDENTE* cui si riferiscono; ciò è quanto accadeva nel modello gerarchico dei dati).

Organizzazione dei record in blocchi

I record di un file devono essere allocati in blocchi (**record blocking**) in quanto il blocco è l'unità di trasferimento dati tra disco e memoria principale.

Quando la dimensione del blocco è superiore a quella del singolo record, si ha che un blocco contiene più record.

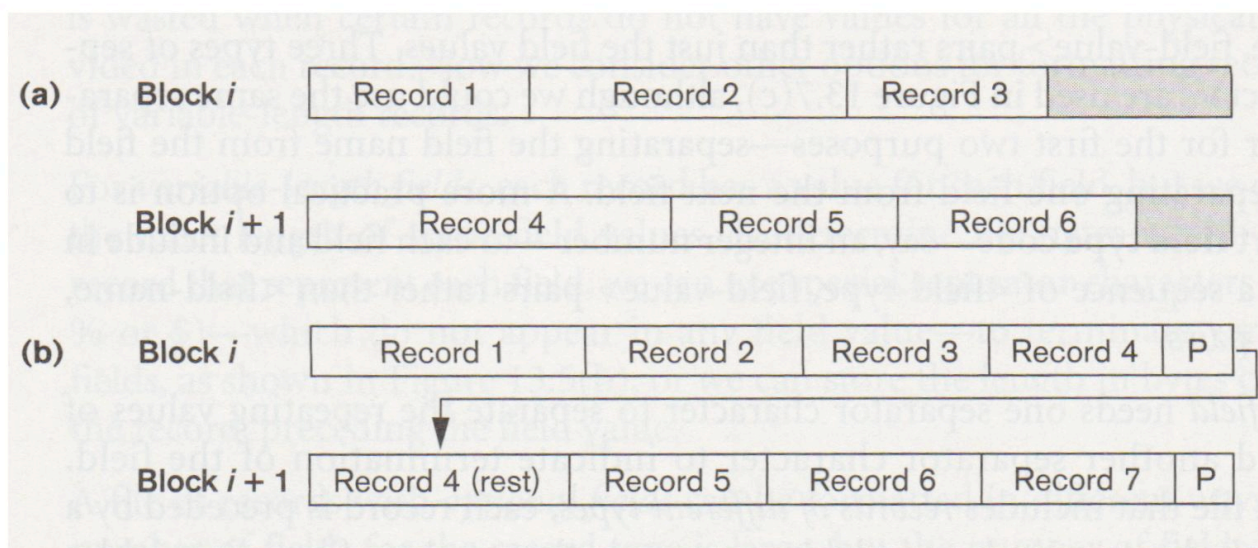
Supponiamo che la dimensione del blocco sia pari a B **byte**.

Assumendo che un dato file sia composto da record di dimensione fissa pari a R **byte**, con $B \geq R$, in ogni blocco possono essere inseriti bfr record (**blocking factor**):

$$bfr = \left\lfloor \frac{B}{R} \right\rfloor$$

Esempio: se $B = 120$ e $R = 15$, allora $bfr = 8$

Record spanned e unspanned



Organizzazione unspanned (a) e spanned (b) dei record

Dimensionamento dei file

Noti la dimensione del blocco, la dimensione del record e il numero di record contenuti nel file (tale numero può ovviamente variare nel tempo; occorre, quindi, disporre di una stima attendibile del numero medio/massimo di record del file), è possibile determinare il numero n_b di blocchi necessari per memorizzare il file.

Assumiamo un'organizzazione dei record di tipo *unspanned*. Siano r e bfr il numero di record del file e il blocking factor, rispettivamente. Il **numero di blocchi** necessari è pari a:

$$n_b = \left\lceil \frac{r}{bfr} \right\rceil$$

File header

Un **file header** (o descrittore del file) contiene le informazioni necessarie per determinare gli indirizzi sul disco dei blocchi del file.

Descrizioni relative al formato dei record:

- record di lunghezza fissa (e unspanned): lunghezza dei campi e ordine dei campi nel record;
- record di lunghezza variabile: codici tipo dei campi, caratteri di separazione, codici tipo dei record.

Ricerca di record su disco

La ricerca di un record su disco prevede di copiare uno o più blocchi nel buffer della memoria principale. Specifici programmi cercano all'interno dei buffer il/i record desiderati, utilizzando l'informazione contenuta nel file header.

Se l'indirizzo del blocco che contiene il record voluto non è noto, i programmi di ricerca devono eseguire una **ricerca lineare** attraverso i blocchi dei file: ogni blocco del file è copiato nel buffer e la ricerca prosegue sino a quando o viene individuato il record desiderato o sono stati controllati tutti i record del file. Tale ricerca risulta estremamente dispendiosa per file di notevole lunghezza.

Operazioni sui file

Le operazioni sui file possono essere suddivise in due categorie:

- operazioni di recupero dei dati (ricerca)
- operazioni di aggiornamento

Le **operazioni di ricerca** non modificano alcun dato presente nel file, ma si limitano a localizzare alcuni record del file in modo da poter esaminare ed eventualmente elaborare il contenuto di alcuni dei loro campi.

Le **operazioni di aggiornamento** modificano il file, inserendo o cancellando record o modificando il valore di alcuni dei loro campi.

In entrambi i casi, è necessario selezionare uno o più record sulla base di un'opportuna **condizione di selezione**.

Organizzazione dei file e metodi di accesso

Organizzazione dei file: organizzazione dei dati contenuti in un file in record, blocchi e strutture di accesso (fanno parte dei criteri di organizzazione di un file le modalità secondo le quali i record e i blocchi sono memorizzati e collegati fra loro)

Metodi di accesso: insieme di programmi che permettono di effettuare operazioni sui file. Ogni metodo di accesso può essere applicato solo a file organizzati in modo opportuno. Ad esempio, un metodo di accesso basato su indice non può ovviamente essere applicato ad un file che non è organizzato attraverso indici.

File di record non ordinati (heap file)

Nel caso più semplice di organizzazione, i record sono posizionati nel file nell'ordine in cui vengono inseriti: ogni nuovo record viene inserito alla fine del file.

Un file organizzato in questo modo è detto **heap file**.

L'inserimento di un nuovo record può essere effettuato in modo molto efficiente:

- (a) l'ultimo blocco del file (su disco) è copiato in un buffer;
- (b) il nuovo record viene aggiunto nel blocco contenuto nel buffer;
- (c) il blocco aggiornato viene riscritto su disco.

L'indirizzo dell'ultimo blocco del file viene mantenuto nel file header.

Ricerca in un heap file

La ricerca di un record in un heap file richiede una **ricerca lineare** su file blocco per blocco (procedura computazionalmente molto costosa).

Se un solo record soddisfa la condizione di ricerca, mediamente dovrà essere controllato almeno il 50% dei blocchi del file (su disco).

Per un file di n_b blocchi, il programma dovrà controllare $n_b/2$ blocchi nel caso medio; se nessun blocco soddisfa la condizione di ricerca, il programma dovrà controllare n_b blocchi.

Cancellazione in un heap file

Per **cancellare** un record, il programma deve effettuare le seguenti operazioni: (i) cercare il record; (ii) copiare il blocco che lo contiene in un buffer; (iii) cancellare il record dal buffer; (iv) riscrivere il blocco sul disco.

In generale, ogni cancellazione crea dello spazio inutilizzato nel blocco su disco. La cancellazione di molti record causa la perdita di quantità significative di spazio (wasted space).

Un'altra tecnica per cancellare record utilizza un extra byte o bit (deletion marker) associato ad ogni record. Il record è cancellato (logicamente) assegnando al deletion marker un certo valore. Un valore diverso dal deletion mark indica che il record è valido. Tale tecnica richiede, però, una riorganizzazione periodica del file per recuperare dello spazio di memoria.

Lettura ordinata in un heap file

Lettura ordinata. Per leggere tutti i record secondo l'ordine definito dai valori di uno o più campi, viene creata una copia ordinata (sorted) del file.

L'ordinamento del file è computazionalmente oneroso per file di grandi dimensioni:

- (a) i record contenuti in ciascun blocco vengono ordinati;
- (b) coppie di blocchi ordinati vengono unite per creare gruppi (run) di record ordinati (di dimensione pari a 2 blocchi);
- (c) run di due blocchi vengono a loro volta uniti per formare run di 4 blocchi; tale operazione viene ripetuta fino a quando si ottiene il run finale che coincide con il file ordinato.

File di record ordinati (file ordinati)

E' possibile ordinare fisicamente i record di un file su disco sulla base dei valori di uno (o più) dei loro campi (**ordinamento dei campi**).

	Name	Ssn	Birth_date	Job	Salary	Sex
Block 1	Aaron, Ed					
	Abbott, Diane					
	⋮					
	Acosta, Marc					
Block 2	Adams, John					
	Adams, Robin					
	⋮					
	Akers, Jan					
Block 3	Alexander, Ed					
	Alfred, Bob					
	⋮					
	Allen, Sam					

Vantaggi dei file ordinati

I file ordinati presentano degli ovvi vantaggi:

- (a) la lettura di tutti i record secondo l'ordine definito dal campo ordinante risulta molto efficiente (non è richiesto alcun ordinamento);
- (b) la ricerca del record successivo al record corrente è efficiente perché non richiede accessi addizionali a blocco (il record successivo e il record corrente si trovano nello stesso blocco, a meno che il record corrente non sia l'ultimo record del blocco);
- (c) una condizione di ricerca basata sul valore del campo (chiave) ordinante può essere verificata in modo efficiente perché può essere utilizzata la **ricerca binaria** (anziché la ricerca lineare).

File ordinati e ricerca binaria

La ricerca binaria viene effettuata a livello di blocchi, non di record.

Supponiamo di avere un file con n_b blocchi, numerati da 1 a n_b , i cui record siano ordinati in modo crescente sulla base del valore che assumono sul campo (chiave) ordinante (ordering key field), e di voler individuare un record con valore del campo (chiave) ordinante uguale a k .

Una ricerca binaria usualmente accede a $\log_2 n_b$ blocchi, indipendentemente dal fatto che il record sia presente o meno, contro $n_b/2$ accessi (nel caso medio), qualora il record sia presente, e n_b accessi, qualora il record sia assente, nel caso della ricerca lineare

Ricerca, inserimento e modifica

Una condizione di ricerca che coinvolge gli operatori di confronto $>$, $<$, $\geq$, e $\leq$ su un campo (chiave) ordinante risulta molto efficiente in quanto tutti i record che soddisfano la condizione sono fra loro contigui nel file.

Esempio. La condizione di ricerca $NAME < 'F'$, dove $<$ denota la relazione di ordinamento alfabetico, seleziona tutti i record compresi fra l'inizio del file e il primo record con valore del campo $NAME$ che inizia con la lettera F .

Le operazioni di **inserimento** e **modifica** di record sono operazioni costose in quanto occorre preservare l'ordinamento (fisico) del file.

Inserimento di un record

L'**inserimento** di un record richiede di:

- (a) cercare la posizione in cui il nuovo record va inserito;
- (b) creare lo spazio necessario per il posizionamento del nuovo record (occorre mediamente spostare metà dei record del file, ossia leggere e riscrivere metà dei blocchi del file).

Possibili soluzioni:

- (i) mantenere in ogni blocco dello spazio libero per gli eventuali inserimenti di nuovi record;
- (ii) creare un file temporaneo non ordinato (file di overflow o transaction file). I nuovi record vengono inseriti alla fine del file di overflow. Tale file viene periodicamente unito al file principale attraverso un'operazione di riorganizzazione.

Modifica di un record

Le modalità di esecuzione della **modifica** del valore di un campo di un record dipendono da due fattori:

- (i) la condizione di ricerca del record da modificare;
- (ii) il campo da modificare.

Se la condizione di ricerca utilizza il campo (chiave) ordinante, è possibile localizzare il record mediante una ricerca binaria; altrimenti, va effettuata una ricerca lineare.

Considerazione conclusiva. I file ordinati sono raramente utilizzati nell'ambito delle basi di dati per applicazioni reali, a meno che non sia disponibile una struttura di indicizzazione (indice primario) che consenta di definire un opportuno cammino per un accesso efficiente ai dati.

Tecniche di hashing

Un altro tipo di organizzazione primaria dei file è quello basato sulle tecniche di **hashing** che consente un accesso molto veloce ai record rispetto a determinate condizioni di ricerca (*hash* o *direct file*).

La **condizione di ricerca** è tipicamente una condizione di uguaglianza su un singolo campo (*hash field del file*); tale campo è spesso anche un campo chiave del file (*hash key*).

Funzionamento (**hashing esterno**): viene utilizzata una funzione h (*hash o randomizing function*) che, applicata al valore del campo hash di un record, restituisce l'indirizzo del blocco su disco in cui il record è contenuto.

Le tecniche di hashing si possono utilizzare anche per manipolare file di record temporanei, di piccole dimensioni, all'interno di un programma (*hashing interno*).

Hashing interno

Per i file interni, la tecnica di hashing è implementata per mezzo di array di record.

Funzionamento (**hashing interno**): supponiamo che l'indice dell'array vari da 0 a $m - 1$; si hanno m slot il cui indirizzo corrisponde all'indice dell'array.

Si usa una funzione di hash che trasforma il valore del campo hash in un numero intero compreso tra 0 e $m - 1$. Ad esempio, se il campo hash è di tipo intero,

$$h(k) = k \text{ MOD } m$$

restituisce il valore intero, compreso tra 0 e $m - 1$, del resto della divisione k/m (*indirizzo del record*).

Un esempio di hashing interno - 1

Si consideri una tabella *IMPIEGATO* con campi *NAME*, *SSN*, *JOB* e *SALARY*.

	NAME	SSN	JOB	SALARY
0				
1				
2				
3				
	⋮			
$M-2$				
$M-1$				

Un esempio di hashing interno - 2

Array di m posizioni per internal hashing

$$h(SSN) = SSN \text{ MOD } m$$

Valori del campo hash non interi possono essere trasformati in interi prima di applicare loro la funzione *MOD*.

Nel caso di stringhe di caratteri, ad esempio, la trasformazione può far uso del codice numerico associato ai singolo caratteri (si noti come, nell'esempio del lucido successivo, parole che sono l'una una permutazione dell'altra generino lo stesso indirizzo di hash).

Problema

La maggior parte delle funzioni di hashing non garantisce che a valori distinti del campo hash corrispondano indirizzi distinti, poiché l'insieme dei possibili valori del campo hash è molto più grande dello spazio degli indirizzi.

Quando l'indirizzo determinato dalla funzione di hashing in corrispondenza del valore del campo hash di un nuovo record coincide con quello già assegnato ad un altro record si genera una **collisione**.

In tal caso, bisogna determinare un'altra posizione nella quale inserire il nuovo record; il processo di ricerca di tale posizione è chiamato **risoluzione delle collisioni** (collision resolution).

Tecniche di risoluzione delle collisioni - 1

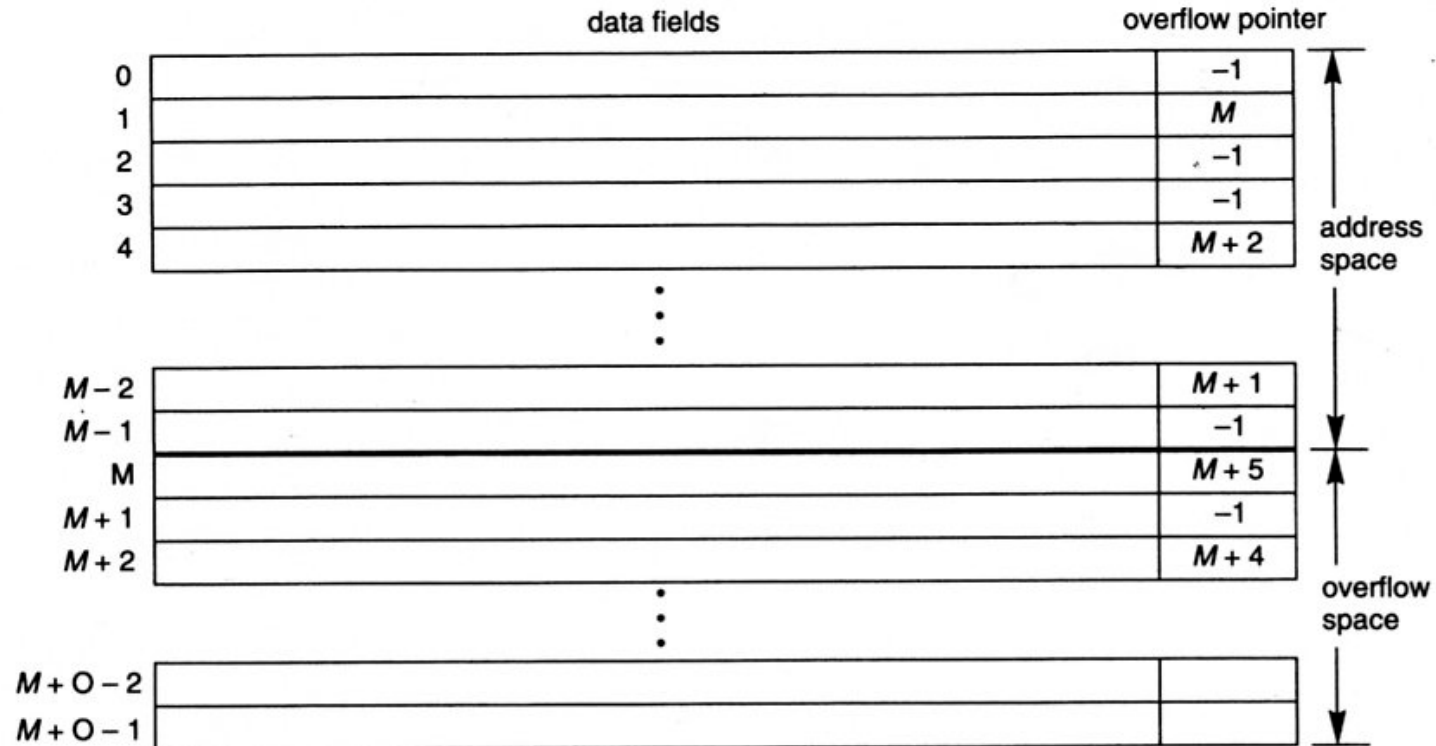
Open addressing. Il programma, partendo dalla posizione (occupata) determinata dalla funzione di hashing, controlla le posizioni successive finché non trova una posizione libera:

Tecniche di risoluzione delle collisioni - 2

Multiple hashing. Questo metodo utilizza una seconda funzione di hash se l'applicazione della prima ha prodotto una collisione. Se si verifica una seconda collisione, o si utilizza il metodo di open addressing o si applica una terza funzione di hash, e così via.

Chaining. Questo metodo considera diverse locazioni di overflow, estendendo l'array con un certo numero di posizioni. Inoltre, esso aggiunge un campo puntatore ad ogni posizione dell'array. Ogni collisione è risolta posizionando il nuovo record in una locazione di overflow non utilizzata e settando il puntatore dell'ultima posizione occupata incontrata all'indirizzo della locazione di overflow (vedi la figura alla pagina successiva).

Tecniche di risoluzione delle collisioni - 3



- null pointer = -1.
- overflow pointer refers to position of next record in linked list.

Soluzione ottimale

L'obiettivo di una funzione di hashing è quello di distribuire i record uniformemente nello spazio degli indirizzi, in modo da minimizzare le collisioni e non lasciare troppe locazioni inutilizzate.

Simulazioni e studi hanno dimostrato che la **soluzione ottimale** è quella di avere una hash table completa dal 70% al 90%, in modo che il numero di collisioni rimanga basso e, al tempo stesso, non si sprechi troppo spazio.

Esempio. Se immaginiamo di dover memorizzare r record nella tabella, dobbiamo utilizzare uno spazio degli indirizzi comprendente m locazioni in modo che il rapporto r/m sia compreso tra 0.7 e 0.9.

Alcune funzioni di hash richiedono che m sia una potenza di 2.

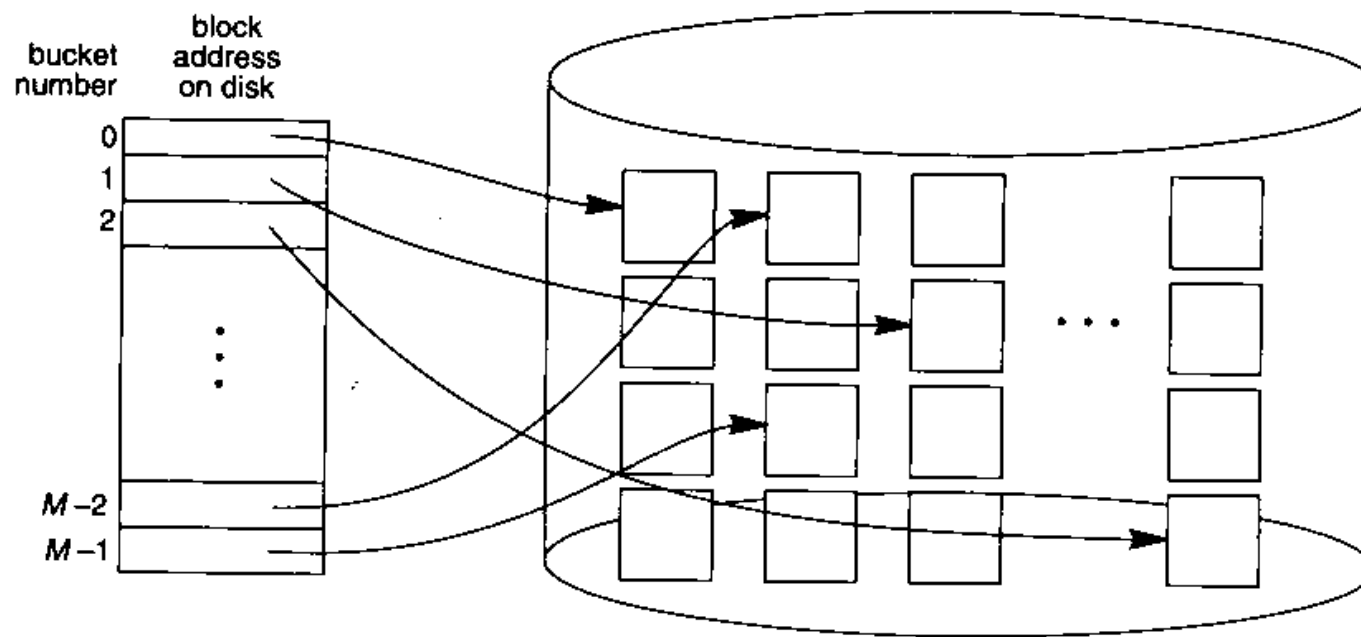
External hashing - 1

La tecnica di hashing per file che risiedono su disco (disk file) è chiamata **external hashing**.

Date le caratteristiche della memorizzazione su disco, lo spazio degli indirizzi è composto di celle (bucket), ognuna delle quali contiene più record. Una cella può essere costituita sia da un singolo blocco che da un insieme di blocchi contigui.

Una tabella contenuta nel file header converte il numero di cella nel corrispondente indirizzo del blocco sul disco (vedi figura).

External hashing - 2



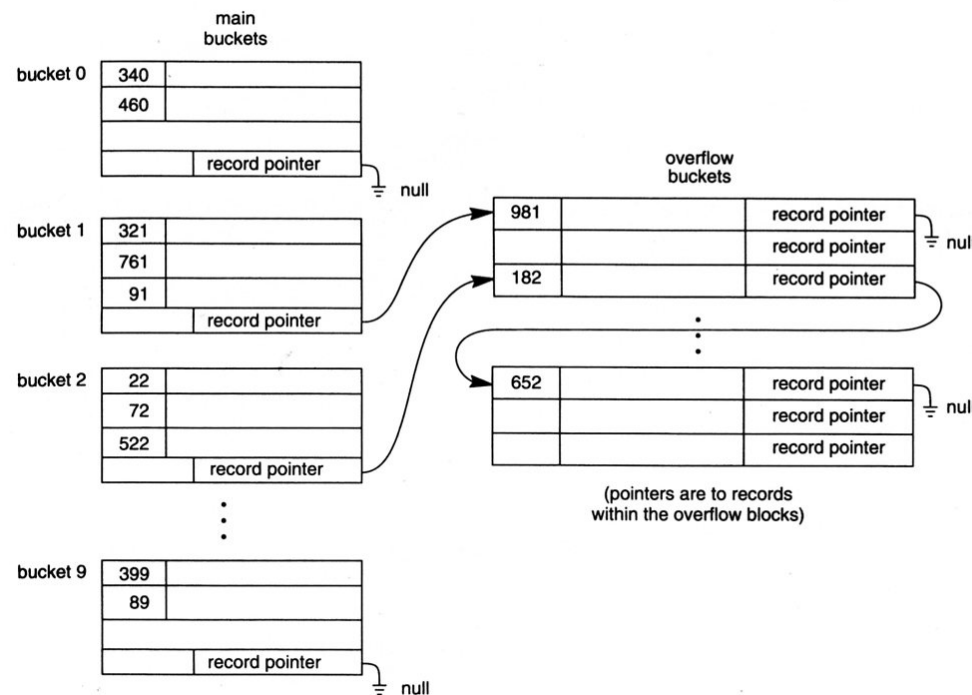
External hashing - 3

Il problema delle collisioni è meno grave rispetto all'internal hashing, poiché molti record possono essere contenuti nella stessa cella.

Insorgono dei problemi solo quando una cella è piena e un nuovo record deve essere inserito in quella cella.

In tal caso, si usa una variante del metodo di chaining visto in precedenza, in cui in ogni cella si mantiene un puntatore ad una lista di record di overflow per quella cella (vedi figura).

External hashing - 4



N.B. Un record pointer include sia l'indirizzo di un blocco che la posizione relativa del record all'interno del blocco.

Un limite significativo

Un limite rilevante delle tecniche di hashing è che lo spazio allocato per un file è fisso.

Supponiamo di voler allocare M celle per lo spazio degli indirizzi e sia m il numero massimo di record che possono essere contenuti in una cella; ne segue che, al massimo, potranno essere allocati $m \cdot M$ record.

Se il numero di record risulta $\ll m \cdot M$ si avrà una grossa quantità di spazio inutilizzato, mentre se sarà $\gg m \cdot M$ si avranno numerose collisioni (e le operazioni di retrieval saranno notevolmente rallentate).

Soluzione. Cambiare il numero di blocchi allocati e, quindi, utilizzare una diversa funzione di hashing per ridistribuire i record nelle varie celle.

Conseguenza: il numero di celle deve variare dinamicamente.

Vengono usate tecniche di hashing con **espansione dinamica** del file.

Espansione dinamica del file

Esistono (almeno) **tre tecniche di hashing** diverse che permettono di espandere dinamicamente i file:

- dynamic hashing;
- extendible hashing;
- linear hashing.